Distributions: Histograms, Density & Ridgeline

Workshop 3 — Module 1

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

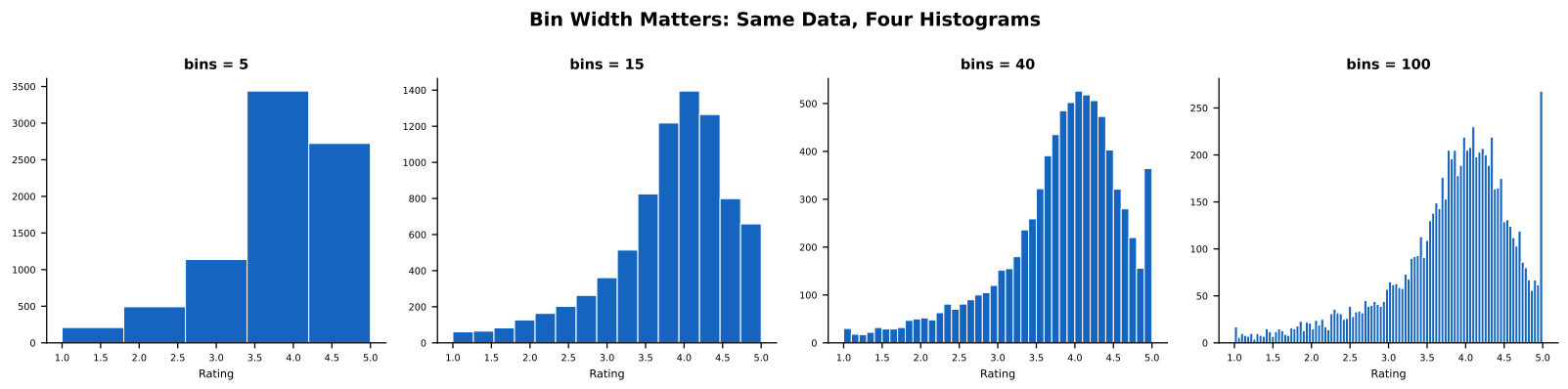
Module Roadmap

1. Histograms: bin width selection and its consequences
2. Histogram variants: frequency, density, cumulative
3. Kernel Density Estimation: theory and bandwidth
4. Histogram + density overlay
5. Grouped densities and ridgeline plots
6. Distribution chart selection guide

Learning Outcome
You will choose the correct distribution chart for any dataset, understand why bin width and bandwidth matter, and produce ridgeline plots for multi-group comparisons.

Histograms

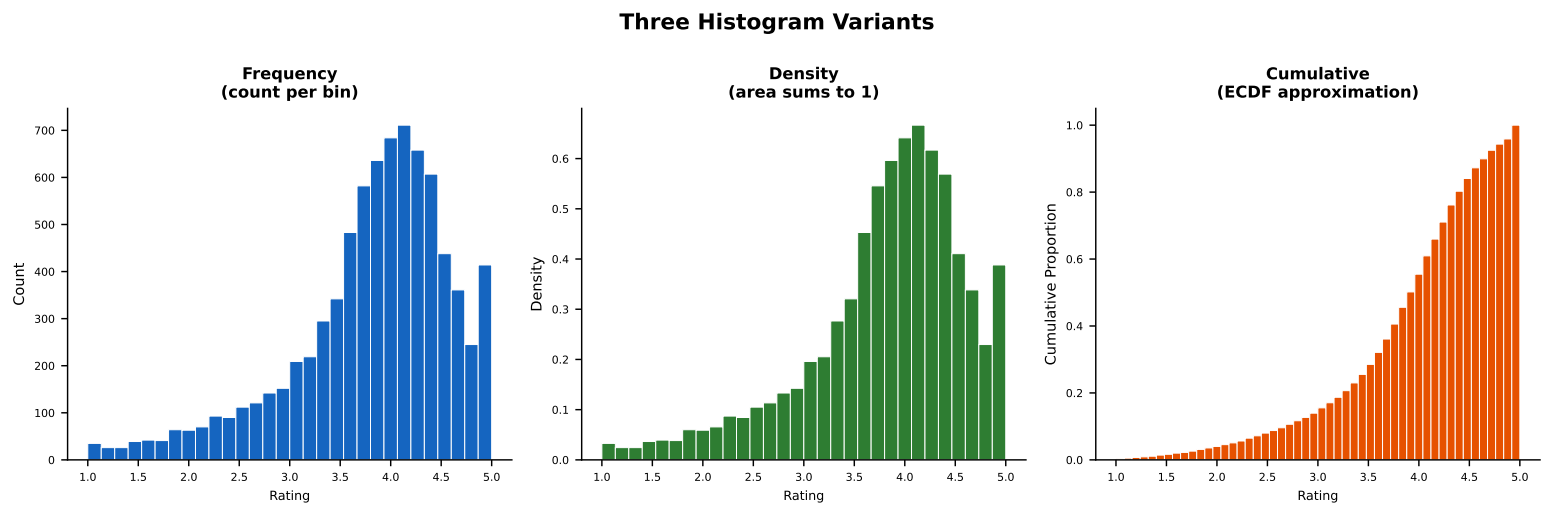
Bin Width Matters



Pro-Tip

Too few bins hides structure; too many creates noise. Sturges' rule ($k = 1 + \log_2 n$) and Freedman-Diaconis ($h = 2 \cdot IQR \cdot n^{-1/3}$) provide starting points. Always try 2–3 values and inspect.

Three Histogram Variants



Data Insight

Frequency: raw counts, intuitive but scale-dependent. **Density**: area sums to 1, comparable across sample sizes. **Cumulative**: shows percentile structure ("50% of apps are rated below 4.2").

Histograms in R vs Python

R (ggplot2)

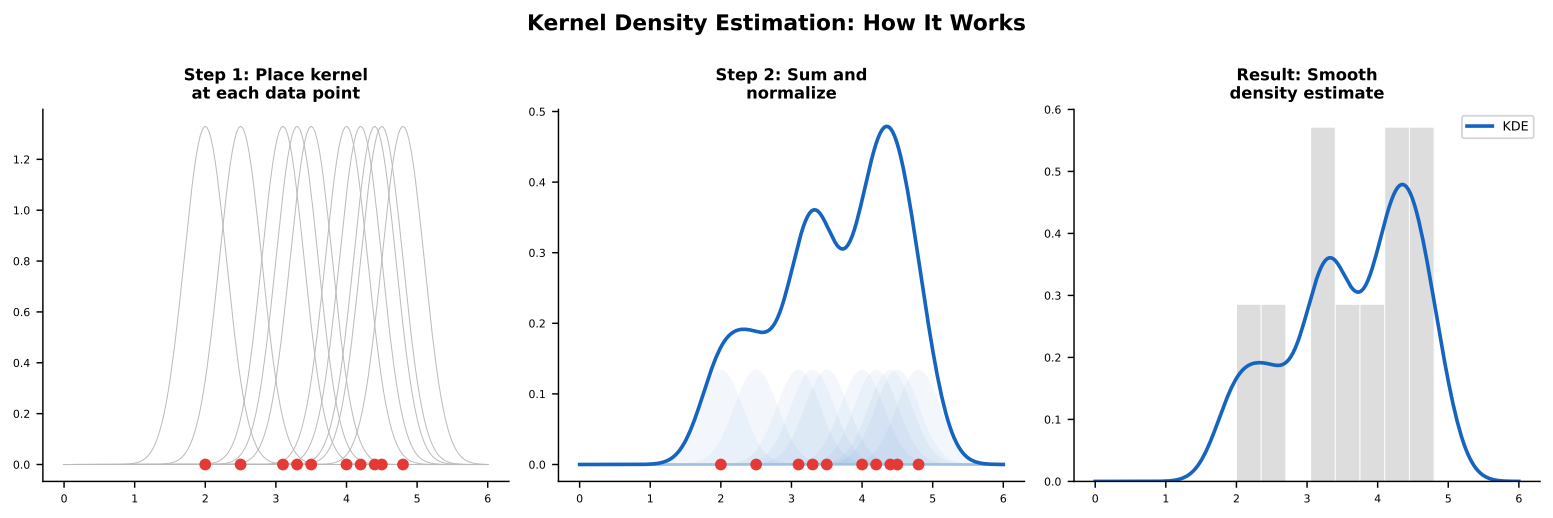
```r
# Frequency histogram
ggplot(apps, aes(x = Rating)) +
  geom_histogram(bins = 30,
    fill = "#1565C0",
    color = "white") +
  theme_minimal()

# Density histogram (area = 1)
ggplot(apps, aes(x = Rating)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 30, fill = "#1565C0",
    color = "white") +
  theme_minimal()

# Cumulative (ECDF)
ggplot(apps, aes(x = Rating)) +
  stat_ecdf(color = "#E65100",
            linewidth = 1) +
  theme_minimal()
```

Python

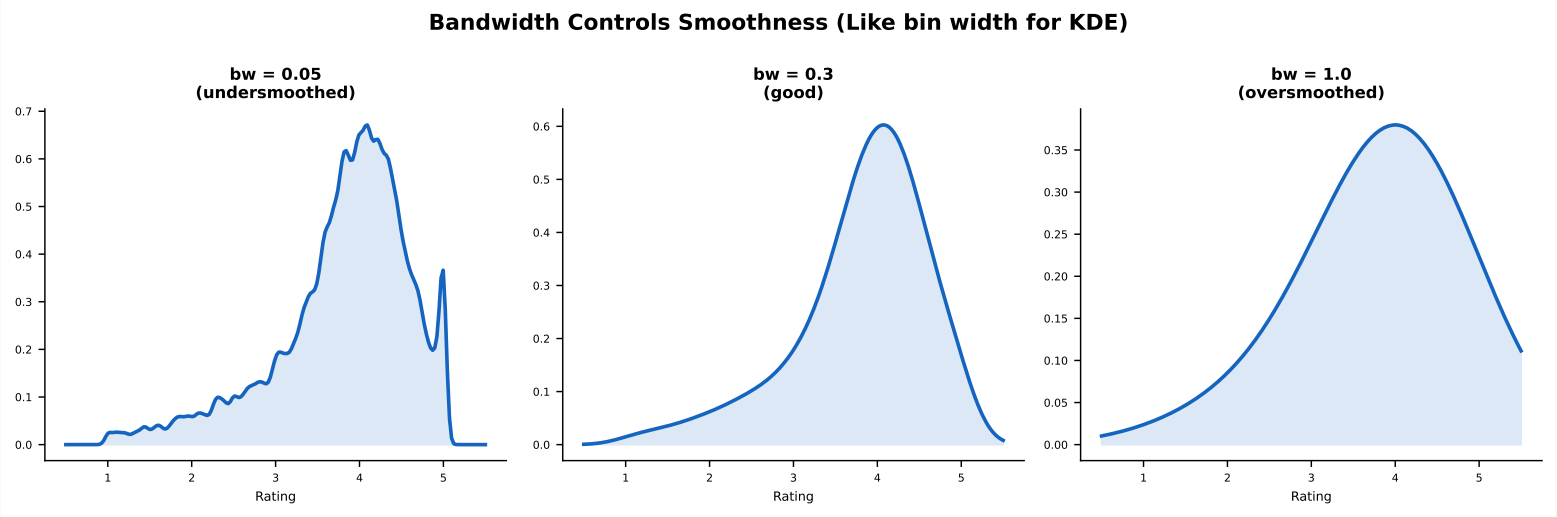
```python
# seaborn (recommended)
sns.histplot(apps, x="Rating",
    bins=30, kde=False)

# Density histogram
sns.histplot(apps, x="Rating",
    bins=30, stat="density")

# ECDF
sns.ecdfplot(apps, x="Rating")

# matplotlib (manual)
ax.hist(ratings, bins=30,
    color="#1565C0",
    edgecolor="white")

# Density
ax.hist(ratings, bins=30,
    density=True)

# Cumulative
ax.hist(ratings, bins=50,
    cumulative=True, density=True
        )
```

Kernel Density Estimation

KDE: How It Works



Bandwidth Controls Smoothness



Pro-Tip

Bandwidth is to KDE what bin width is to histograms. Too small (undersmoothed): noise. Too large (oversmoothed): hides modes. Default bandwidth selectors (Silverman's rule) work well for unimodal data but may oversmooth multimodal distributions.

KDE in R vs Python

R (ggplot2)

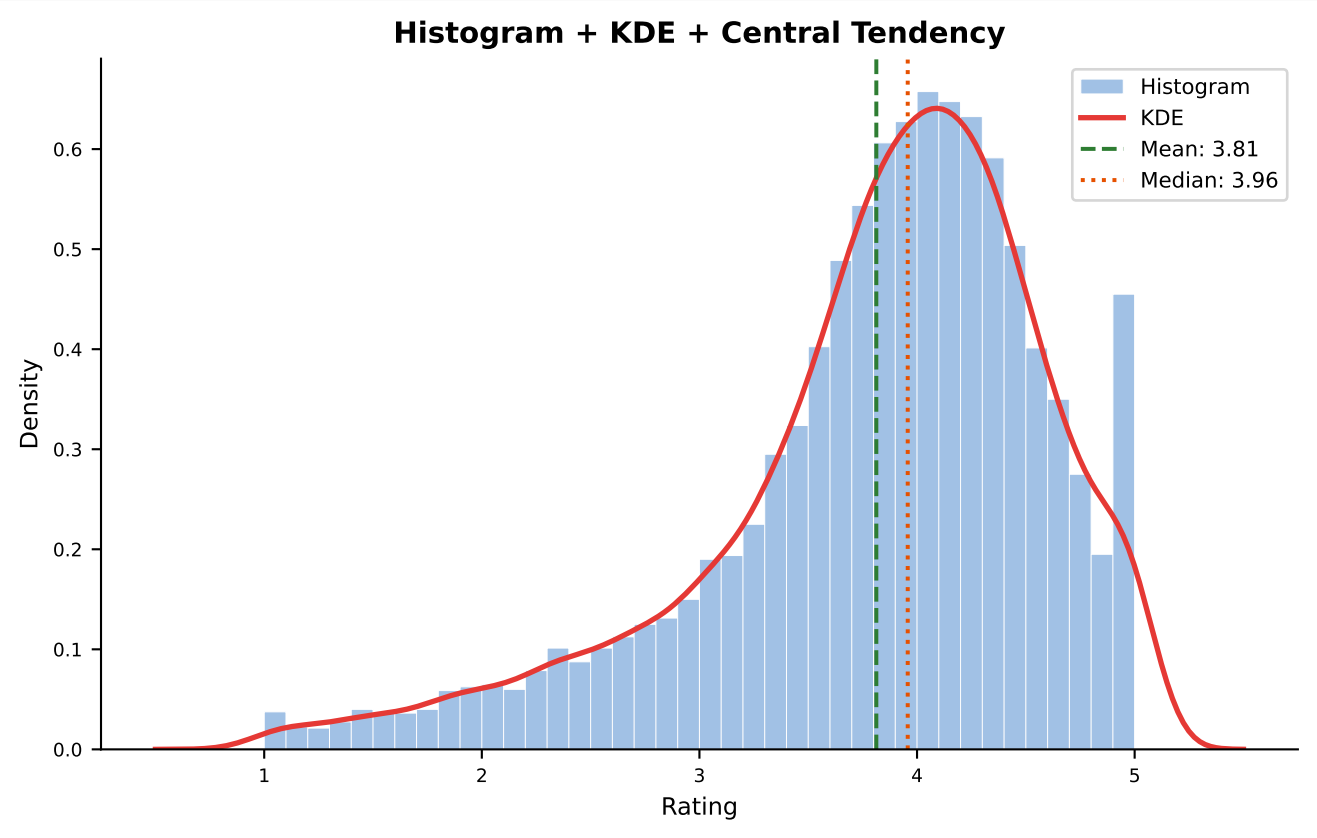
```r
# Density curve
ggplot(apps, aes(x = Rating)) +
  geom_density(fill = "#1565C0",
    alpha = 0.3, linewidth = 1) +
  theme_minimal()

# Adjust bandwidth
ggplot(apps, aes(x = Rating)) +
  geom_density(
    adjust = 0.5,  # half default
          bw
    fill = "#1565C0",
    alpha = 0.3) +
  theme_minimal()

# adjust > 1 = smoother
# adjust < 1 = rougher
```

Python

```python
# seaborn
sns.kdeplot(apps, x="Rating",
    fill=True, alpha=0.3,
    color="#1565C0")

# Adjust bandwidth
sns.kdeplot(apps, x="Rating",
    bw_adjust=0.5,  # half
         default
    fill=True, alpha=0.3)

# scipy (manual)
from scipy.stats import
     gaussian_kde
kde = gaussian_kde(ratings,
    bw_method=0.3)
xs = np.linspace(1, 5, 200)
ax.plot(xs, kde(xs))
ax.fill_between(xs, kde(xs),
    alpha=0.15)
```

Histogram + Density Overlay

The Best of Both: Histogram + KDE + Reference Lines



Data Insight

Overlaying KDE on a density-scaled histogram shows both the binned structure and the smooth shape. Adding mean and median lines reveals skewness: when mean $<$ median, the distribution is left-skewed.

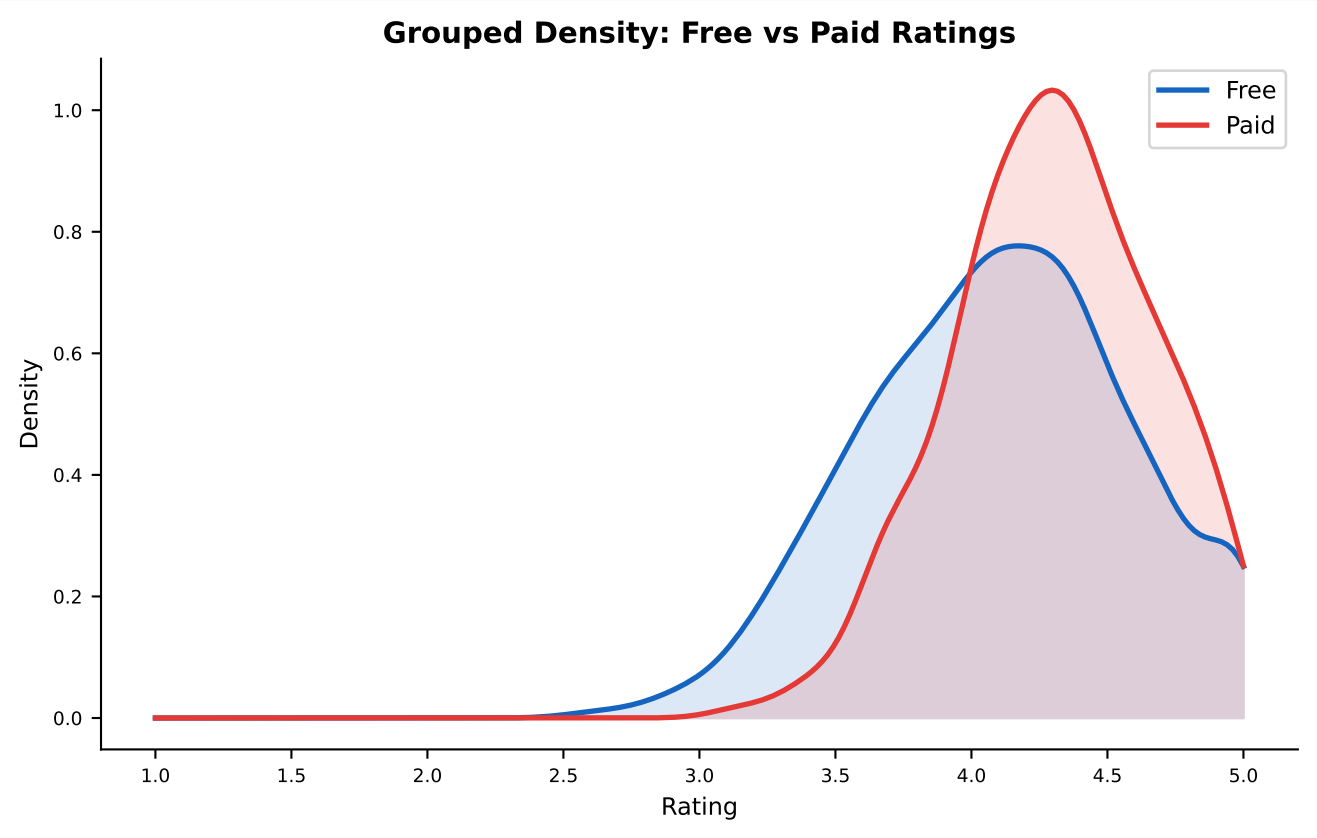
Overlay in R vs Python

```r
ggplot(apps, aes(x = Rating)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 35, fill = "#1565C0",
    color = "white", alpha = 0.5)
          +
  geom_density(color = "#E53935",
    linewidth = 1.2) +
  geom_vline(
    xintercept = mean(apps$Rating
          ,
      na.rm = TRUE),
    linetype = "dashed",
    color = "#2E7D32", linewidth
          = 1) +
  theme_minimal() +
  labs(title = "Histogram + KDE",
       x = "Rating", y = "Density
            ")
```

```python
# seaborn: one line!
sns.histplot(apps, x="Rating",
    bins=35, kde=True,
    stat="density",
    color="#1565C0", alpha=0.5)

# Or matplotlib manual:
ax.hist(ratings, bins=35,
    density=True, color="#1565C0"
          ,
    edgecolor="white", alpha=0.5)
kde = gaussian_kde(ratings)
xs = np.linspace(1, 5, 200)
ax.plot(xs, kde(xs),
    color="#E53935", lw=2)
ax.axvline(x=np.mean(ratings),
    color="#2E7D32", lw=1.5,
    linestyle="--")
```

Grouped Density & Ridgeline

Grouped Density: 2–3 Groups

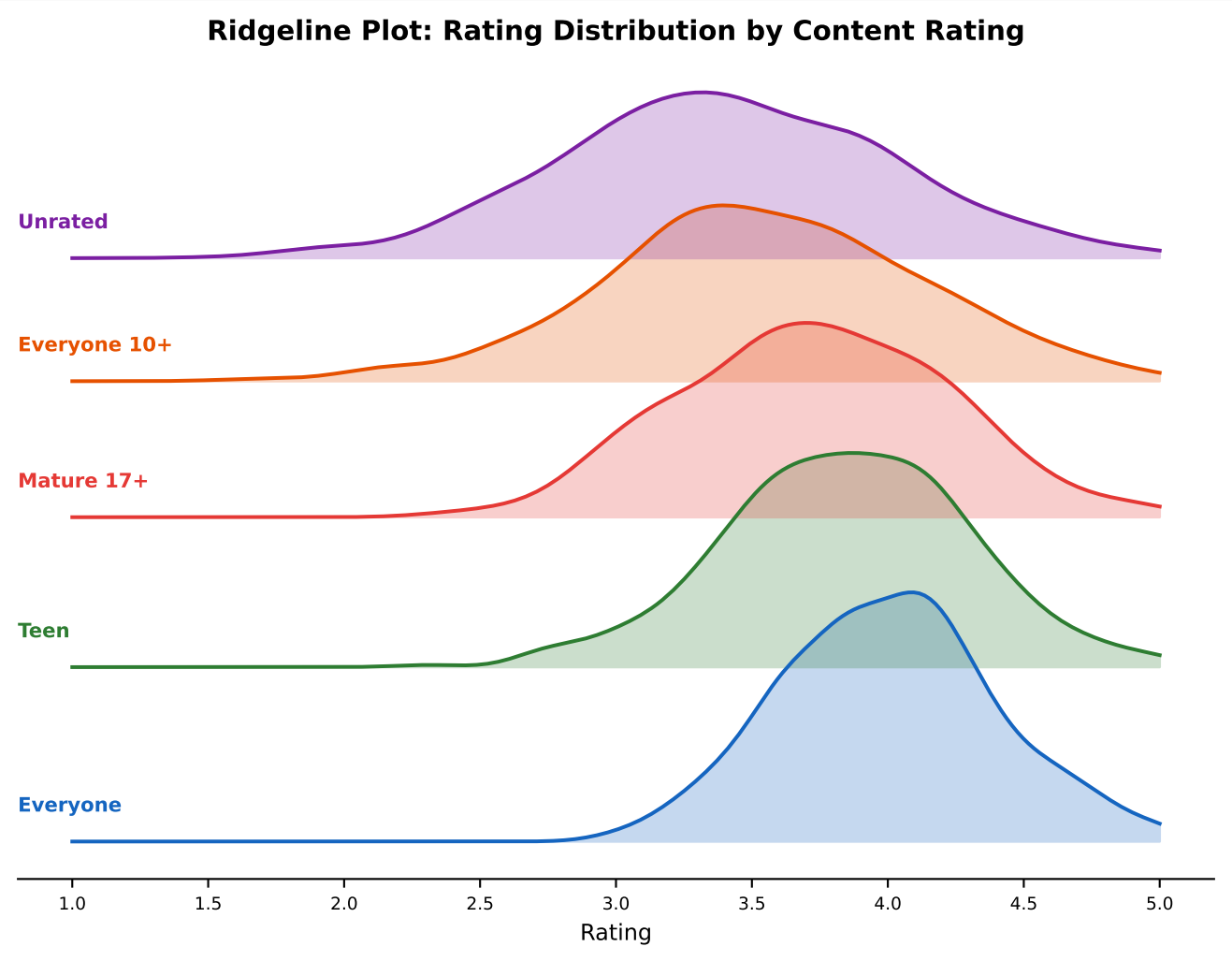


Grouped Density in Code

```r
ggplot(apps, aes(x = Rating,
    fill = Type, color = Type)) +
  geom_density(alpha = 0.2,
    linewidth = 1) +
  scale_fill_manual(values =
    c(Free = "#1565C0",
      Paid = "#E53935")) +
  scale_color_manual(values =
    c(Free = "#1565C0",
      Paid = "#E53935")) +
  theme_minimal() +
  labs(title = "Free vs Paid")
```

```python
# seaborn
sns.kdeplot(data=apps, x="Rating"
    ,
    hue="Type", fill=True,
    alpha=0.2, linewidth=1.5,
    palette={"Free":"#1565C0",
             "Paid":"#E53935"})

# matplotlib manual
for t, c in [("Free","#1565C0"),
             ("Paid","#E53935")]:
    sub = apps.query("Type==@t")
    kde = gaussian_kde(
        sub["Rating"].dropna())
    ax.fill_between(xs, kde(xs),
        alpha=0.15, color=c)
    ax.plot(xs, kde(xs),
        color=c, lw=1.5, label=t)
```

Ridgeline Plot: 4+ Groups



Pro-Tip
Ridgeline plots (joy plots) stack densities vertically, using slight overlap for compactness. In R: `ggridges::geom_density_ridges()`. In

Ridgeline in R

```r
library(ggridges)

ggplot(apps, aes(x = Rating,
                 y = `Content Rating`,
                 fill = `Content Rating`)) +
  geom_density_ridges(
    alpha = 0.4,
    scale = 1.5,          # overlap amount
    rel_min_height = 0.01 # trim tails
  ) +
  scale_fill_brewer(palette = "Set2") +
  theme_minimal() +
  theme(legend.position = "none") +
  labs(title = "Rating by Content Rating",
       x = "Rating", y = NULL)
```

Distribution Selection

Chart Selection Guide

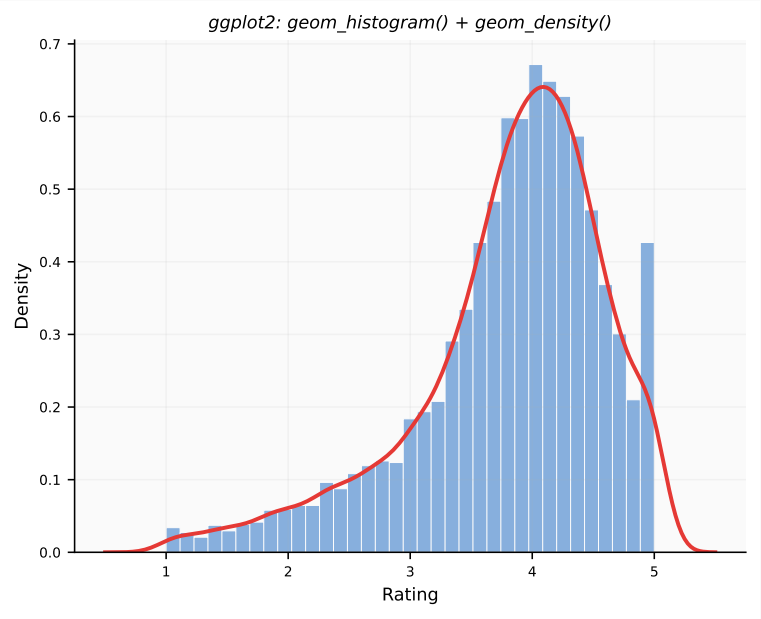
Distribution Chart Selection Guide

Need	Chart	R (ggplot2)	Python
1 variable, shape	**Histogram**	`geom_histogram(bins=30)`	`ax.hist(bins=30)`
1 variable, smooth	**Density (KDE)**	`geom_density(fill='blue')`	`ax.plot(kde(xs))`
1 var + groups (≤3)	**Overlaid density**	`geom_density(aes(fill=g))`	`loop + kde per group`
1 var + groups (4+)	**Ridgeline**	`ggridges::geom_density_ridges()`	`loop + vertical offset`
Hist + density	**Overlay**	`geom_histogram() + geom_density()`	`ax.hist(density=T) + plot`
Cumulative	**ECDF**	`stat_ecdf()`	`ax.hist(cumulative=True)`

Complete Examples

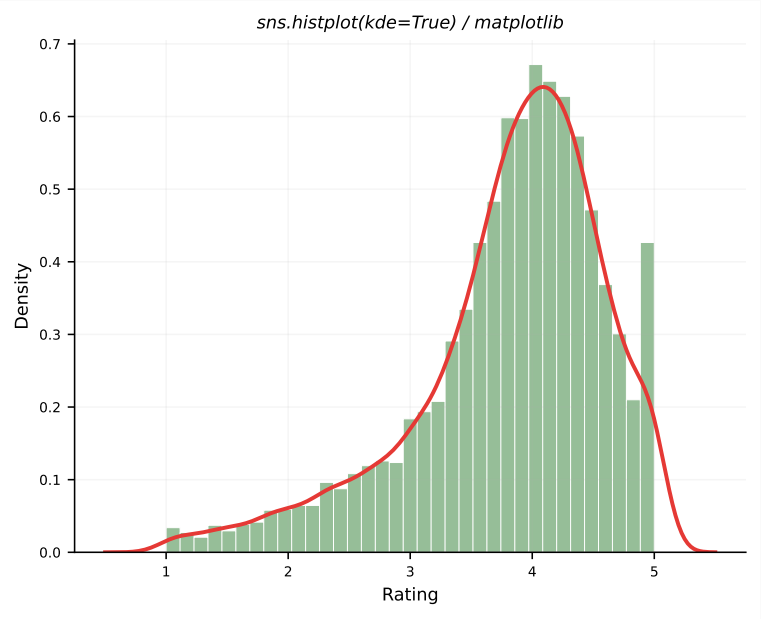
R: Publication-Quality Distribution Chart

```r
ggplot(apps |>
    filter(!is.na(Rating)),
  aes(x = Rating)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 35, fill = "#1565C0",
    color = "white", alpha = 0.5) +
  geom_density(color = "#E53935",
    linewidth = 1.2) +
  geom_vline(
    xintercept = mean(
      apps$Rating, na.rm = TRUE),
    linetype = "dashed",
    color = "#2E7D32") +
  annotate("text",
    x = 3.5, y = 1.2,
    label = "Left-skewed:\nhigh-
        rating mode",
    fontface = "italic",
    color = "#555", size = 3) +
  theme_minimal() +
  labs(title = "App Ratings are
      Left-Skewed",
    subtitle = "Google Play Store,
                n = 9,366",
    x = "Rating", y = "Density")
```



Python: Same Chart

```
fig, ax = plt.subplots(figsize
    =(6,5))

ax.hist(ratings, bins=35,
    density=True, color="#2E7D32",
    edgecolor="white", lw=0.3,
    alpha=0.5)

kde = gaussian_kde(ratings)
xs = np.linspace(0.5, 5.5, 200)
ax.plot(xs, kde(xs),
    color="#E53935", lw=2)

ax.axvline(x=np.mean(ratings),
    color="#2E7D32", lw=1.5,
    linestyle="--")

ax.set_title("App Ratings are
    Left-Skewed",
    fontweight="bold")
ax.set_xlabel("Rating")
ax.set_ylabel("Density")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(
    False)
```



Synthesis

Key Takeaways

1. **Bin width** determines histogram structure: always try 2–3 values. Use Freedman-Diaconis as a starting point.
2. **KDE** smooths the histogram; **bandwidth** controls smoothness. Default rules work for unimodal data.
3. **Overlay** histogram + KDE + reference lines for the most informative single-variable display.
4. **Grouped density** works for $\leq$3 groups; **ridgeline** for 4+ groups.
5. In R: `geom_histogram`, `geom_density`, `ggridges::geom_density_ridges`. In Python: `sns.histplot(kde=True)`, `sns.kdeplot`, `joypy`/manual offset.

Essential Readings

- Wickham, H. (2016). *ggplot2*, Chapter 5 (Statistical Summaries).
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapters 7–9 (Histograms, Density, Multiple Distributions).
- Silverman, B. W. (1986). *Density Estimation for Statistics and Data Analysis*. Chapman & Hall.

Next Module: Distributions: Boxplots, Violins & Beeswarm (W03-M02)